world where everything is injected, all you see is things being injected. What does this mean and how can you avoid it?

10.3.3. Avoiding dependency injection hell

Imagine you're writing part of the dashboard for Agtron's tracking application that displays the number of visits for a particular user. As part of this, you have a `visits` function with three dependencies: a database, an HTTP service, and a user.

Suppose this `visits` function looks up the user in the database and then uses that information to get the number of hits for the user from the HTTP service:

```
visits = (database, http, user) ->
 http.hitsFor database.userIdFor(user.name)
```

Suppose you then find out that some users have requested their data be kept private and that information be available only in a `permissions` file. You add another argument:

```
visits = (database, http, user, permissions) ->
  if permissions.allowDataFor user
    http.hitsFor database.userIdFor(user.name)
  else
    'private'
```

With more and more dependencies, you can see that this will get more and more unwieldy. An options object passed in as an argument (see chapter 4) is a common solution to the problem of having too many arguments:

```
visits = (dependencies) ->
  if dependencies.permissions.allowDataFor user
    dependencies.http.hitsFor
dependencies.database.userIdFor(user.name)
  else
    'private'
```

That's different . . . and worse! The function is now *harder* to understand.